LangGraph Core Components

A LangGraph application has four major components:

1) State ⇒ It is the shared memory that flows through our workflow. It holds all the data passed between nodes and evolves over time. It is typically defined using either TypedDict or Pydantic classes in python. These are accessible and mutable by every node.

2) Nodes ⇒ It represents a single task in a workflow. Behind the node, every node is a python function. When a node executes, it receives the current state as input, performs its logic, and returns a partial update to the State.

3) Edges ⇒ These define the flow of execution, dictating which node runs next. Edges can be sequential, parallel, or conditional (branching/looping).

4) Reducers ⇒ It defines how updates from nodes are applied to the shared state. By default, returning a value overwrites the existing state value. Reducers allow us to specify if a new value should replace, add to, or merge with the existing value (crucial for maintaining a list of chat messages).

The Execution Model

LangGraph's execution model is inspired by Google Pregel (a system for large-scale graph processing).

1) Definition & Compilation ⇒ You define nodes/edges and compile the graph to check for structural errors (like orphaned nodes).

2) Message Parsing ⇒ The process of passing the updated State from one node to the next via edges.

3) Supersteps ⇒ Execution happens in "Supersteps" rather than simple steps, because a simple iteration might involve multiple nodes executing in parallel. The graph stops executing when there are no active nodes ~~when~~ and no messages parsing through edges.

Scanned with OKEN Scanner

Different types of LLM Workflows

An LLM Workflow is a workflow where multiple tasks depend on LLMs for prompting, reasoning, tool calling, or decision making. Five common LLM workflows are discussed below :-

a) Prompt Chaining ⇒ It involves calling the LLMs sequentially, where the output of one call becomes the input for next.
Eg. Generating an outline from a topic, then generating a blog from outline

b) Routing ⇒ An LLM acts as a decision-maker to route a task to the most capable subsequent node (e.g., a customer support bot routing to tech, sales or refund departments).

c) Parallelisation ⇒ Breaking a task into multiple independent subtasks, executing them simultaneously, and aggregating the results
Eg. Checking a Youtube video for community guidelines, misinformation, and explicit content at the same time.

d) Orchestrator-Worker ⇒ Similar to parallelization, but the subtasks are determined dynamically by an orchestrator LLM based on the user's query (eg. an AI researcher deciding whether to search Google Scholar or Google Search).

e) Evaluator-Optimizer ⇒ An iterative, looping workflow where a 'Generator' creates a draft (like an email or blog) and an 'Evaluator' critiques it. The loop continues until the evaluator is satisfied.

LangGraph offers us multiple basic functions and workflows using which can create the above discussed complex workflows.

1. Sequential Workflows ⇒ A sequential workflow executes tasks in a sequential manner. This is useful when next tasks depends on the output of previous tasks. Prompt Chaining LLM workflows can be built using Sequential Workflows.

2. Parallel Workflows ⇒ A parallel workflow executes multiple tasks simultaneously rather than sequentially. This is useful when tasks do not depend on each others output. The workflow branches into multiple nodes at once and often aggregates their outputs later.

Scanned with OKEN Scanner

State Handling in Parallel Workflows

When building parallel workflows, we must handle carefully the state updates to avoid conflicts (the Invalid Update Error).

⇒ Partial State Updates ⇒ If multiple parallel node returns the entire state object, the LangGraph gets confused about which version of state is the "truth". To fix this, parallel nodes should only return a dictionary containing the specific keys they are updating (partial updates).

⇒ Reducers for Merging Data ⇒ If parallel nodes need to update the exact same key, we cannot just overwrite the key. We must use a Reducer function.

3. Conditional Workflows ⇒ A conditional workflow acts like an 'if-else' statement. It involves branching paths, but unlike parallel workflows where all branches execute, in a conditional workflow, execution goes down only one single branch based on a specific condition. Conditional routing requires a standalone function that evaluates the state and returns the name of next node as a string.

* Command API ⇒ Normally, conditional routing requires us to writing a separate routing function and map it using add_conditional_edges(). The Command API bypasses this. A node can directly return a Command object (eg. Command(goto="nextnode")) to dynamically dictate exactly where the graph should go next. Alongside, dictating the next destination, the Command object allows us to update the graph's State at the exact same time.

This feature is highly recommended for complex, dynamic workflows. Especially multi-agent routing because it allows individual agent (nodes) to autonomously decide who should take over the next task without needing a hardcoded, centralized routing function.

4. Iterative (Looping) Workflows ⇒ An iterative workflow creates a cycle between nodes to continuously refine or improve an item/outcome until a specific condition or quality threshold is met.

To create a loop, we simply use 'add_edge()' to point a node back to previous node in a graph, combined with a conditional edge to eventually break the loop.

Scanned with OKEN Scanner